```cpp
template <typename DatetimeAPI_T, typename Calendar_Impl, typename DAP_Protocol>
struct simplydt::DatetimeAPI {
    using Calendar = CalendricalSystem<Calendar_Impl>;
    using NativeCalendar = Calendar_Impl;
    using Protocol = DatetimeAbstractionProtocol<DAP_Protocol>;
    using Repr_Type = typename Protocol::Underlying_Type;
    using Date = typename Calendar::Date;
    using Time = typename Calendar::Time;
    /* How do we get Duration types here cleanly? */
};
```

```cpp
template <typename Calendar_Impl, typename DAP_Protocol>
struct simplydt::StandardDatetimeAPI
    : public DatetimeAPI<StandardDatetimeAPI  Calendar_Impl  DAP_Protocol>
{
    //
};
```

```cpp
template <concepts::contract_abiding_calendar Calendar_Impl>
struct simplydt::CalendricalSystem {

    using Date = typename Calendar_Impl  Date;
    using Time = typename Calendar_Impl  Time;
    using YearInt_t = typename Date  YearInt_t;
    using DayOfWeek = typename Calendar_Impl  DayOfWeek;
    using Month = typename Calendar_Impl  Month;
    using DatetimeFlags = typename Calendar_Impl  DatetimeFlags;???

    [[nodiscard]] static constexpr CalendarSystem calendar() noexcept {
        return Calendar_Impl::calendar();
    };

    [[nodiscard]] static constexpr bool isSolarCalendar() noexcept {
        return Calendar_Impl::isSolarCalendar();
    };

    [[nodiscard]] static constexpr bool isLunarCalendar() noexcept {
        return Calendar_Impl::isLunarCalendar();
    };

    [[nodiscard]] static constexpr bool isLunisolarCalendar() noexcept {
        return Calendar_Impl::isLunisolarCalendar();
    };

    [[nodiscard]] static constexpr bool isValidYear(const YearInt_t year) noexcept {
        return Calendar_Impl::isValidYear(year);
    };

    [[nodiscard]] static constexpr bool isValidMonth(const uint8_t month) noexcept {
        return Calendar_Impl::isValidMonth(month);
    };

    [[nodiscard]] static constexpr bool isLeapYear(const YearInt_t year) noexcept {
        return Calendar_Impl::isLeapYear(year);
    };

    [[nodiscard]] static constexpr uint8_t getDaysInMonth(const YearInt_t year, const uint8_t month) noexcept {
        return Calendar_Impl::getDaysInMonth(year, month);
    };

    [[nodiscard]] static constexpr uint8_t getWeeksInMonth(const YearInt_t year, const uint8_t month) noexcept {
        return Calendar_Impl::getWeeksInMonth(year, month);
    };

    [[nodiscard]] static constexpr bool isValidDate(const Date& date) noexcept {
        return Calendar_Impl::isValidDate(date);
    };

    [[nodiscard]] static constexpr DayOfWeek getDayOfWeek(const Date& date) noexcept {
        return Calendar_Impl::getDayOfWeek(date);
    };

    [[nodiscard]] static constexpr uint8_t getDaysInYear(const YearInt_t year) noexcept {
        return Calendar_Impl::getDaysInYear(year, month);
    };

    [[nodiscard]] static constexpr const char* getDayOfWeekLiteral(const Date& date) noexcept {
        return Calendar_Impl::getDayOfWeekLiteral(date);
    };

    [[nodiscard]] static constexpr const char* getMonthLiteral(const uint8_t month) noexcept {
        return Calendar_Impl::getMonthLiteral(month);
    };

    [[nodiscard]] static constexpr UnixTimestamp toUnixTimestamp(const Date& date) noexcept {
        return Calendar_Impl::toUnixTimestamp(date);
    };

    [[nodiscard]] static constexpr Date fromUnixTimestamp(const UnixTimestamp& timestamp) noexcept {
        return Calendar_Impl::fromUnixTimestamp(timestamp);
    };

    [[nodiscard]] static constexpr Date fromTimePoint(const SystemTimePoint& time_point) noexcept {
        return Calendar_Impl::fromTimePoint(time_point);
    };
    /* Continue implementation... */

};
```

```cpp
struct simplydt::gregorian::GregorianCalendar final : public CalendricalSystem<GregorianCalendar> {
    static constexpr uint8_t FIRST_DAY_OF_WEEK = 0; ///< Sunday

    [[nodiscard]] static constexpr CalendarSystem calendar() noexcept {
        return CalendarSystem::GREGORIAN;
    };

    [[nodiscard]] static constexpr bool isSolarCalendar() noexcept {
        return true;
    };

    [[nodiscard]] static constexpr bool isLunarCalendar() noexcept {
        return false;
    };

    [[nodiscard]] static constexpr bool isLunisolarCalendar() noexcept {
        return false;
    };

    [[nodiscard]] static uint8_t getDaysInMonth(const YearInt_t year, const uint8_t month) noexcept {
        /* Gregorian implementation here... */
    };
    /* Continue implementation... */
};
```

```cpp
/* What is this type responsible for? */
template <typename DAP_Impl, typename Underlying_T>
struct simplydt::DatetimeAbstractionProtocol {
    using Duration = simplydt::StandardDuration<Underlying_T>;???
    using Underlying_Type = Underlying_T;

    template <typename... ArgTypes>
    [[nodiscard]] static Underlying_Type datetimeInProtocol(ArgTypes...) noexcept;

    [[nodiscard]] static ??? datetimeOutProtocol(const Underlying_Type& representation) noexcept;
};
```